

EKS: Scoping a User's kubeconfig to a Single Namespace

Cluster: "test-cluster" · User: "user-a" · Namespace: "user-a"

Restricting access to a namespace in EKS involves two separate mechanisms: the kubeconfig's default namespace (a convenience setting only) and Kubernetes RBAC combined with an IAM identity mapping (the actual security boundary). Both are covered below.

STEP 0

Prerequisite: user-a's IAM identity

This assumes user-a already has an IAM identity — either an IAM user (`arn:aws:iam::<ACCOUNT_ID>:user/user-a`) or a role they assume. You'll need this ARN for the identity mapping in Step 3. (Find it with `aws iam get-user --user-name user-a` or `aws sts get-caller-identity` run as user-a.)

STEP 1 · RUN AS ADMIN

Create the namespace

```
kubectl create namespace user-a
```

STEP 2 · RUN AS ADMIN

Create a Role and RoleBinding scoped to that namespace

```
# user-a-role.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: user-a
  name: user-a-full-access
rules:
  - apiGroups: [ "", "apps", "extensions", "batch" ]
    resources: [ "*" ]
    verbs: [ "*" ]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: user-a-binding
  namespace: user-a
subjects:
  - kind: User
    name: user-a           # Kubernetes-side username
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: user-a-full-access
  apiGroup: rbac.authorization.k8s.io
```

```
kubectl apply -f user-a-role.yaml
```

Narrow the `resources` / `verbs` lists if user-a should only get read access rather than full control of the namespace.

STEP 3 · RUN AS ADMIN

Map user-a's IAM identity to the Kubernetes username "user-a"

Preferred: EKS access entries (if enabled on the cluster)

```
aws eks create-access-entry \
  --cluster-name test-cluster \
  --principal-arn arn:aws:iam::<ACCOUNT_ID>:user/user-a \
  --username user-a
```

Do not attach an access policy (e.g. `AmazonEKSClusterAdminPolicy`) to this access entry. Leaving it policy-less means EKS defers entirely to the RBAC Role from Step 2 — which is what keeps user-a scoped to just that namespace. Attaching a cluster-wide access policy here overrides or

adds to RBAC and can break the scoping.

Alternative: aws-auth ConfigMap (older clusters without access entries)

```
kubectl edit configmap aws-auth -n kube-system
```

```
mapUsers: |
- userarn: arn:aws:iam::<ACCOUNT_ID>:user/user-a
  username: user-a
  groups: []          # important: leave empty — no system:masters, etc.
```

STEP 4 · RUN AS USER-A

Generate user-a's kubeconfig

```
aws eks update-kubeconfig --name test-cluster --region us-east-1 --alias test-cluster
kubectl config set-context test-cluster --namespace=user-a
```

This requires user-a to have their own AWS credentials configured locally (`aws configure` , SSO, etc.) for that IAM identity — the kubeconfig's `exec` plugin calls STS at request time to mint a token; it doesn't embed a static credential.

What actually enforces the restriction

The `--namespace=user-a` default is convenience only — it just saves user-a from typing `-n user-a` every time. The real boundary is Steps 2 and 3: user-a's Kubernetes identity is "user-a", and "user-a" has a RoleBinding only in the `user-a` namespace and nowhere else. If they run `kubectl get pods -n kube-system` or against any other namespace, RBAC denies it with a "forbidden" error.

Things to double-check afterward

- **Cluster-scoped resources** — `kubectl get nodes` , `kubectl get namespaces` , `kubectl get pv` — aren't covered by a namespaced Role at all. user-a will get "forbidden" on these too unless you separately grant a ClusterRole, which you likely don't want here.
- **No overlapping broader access** — make sure user-a isn't mapped anywhere else with wider permissions: another `mapUsers` / `mapRoles` entry, an access entry with a cluster policy attached, or membership in a group like `system:masters` . Any of these would override the narrow scoping regardless of what Step 2 configured.